

## Primitive Data Types

- **char / byte** → 1 byte
- **short** → 2 bytes
- **int** → 4 bytes
- **long** → 4 bytes (32-bit) / 8 bytes (64-bit Linux/Unix)
- **long long** → 8 bytes

## Floating-Point Types

- **float** → 4 bytes
- **double** → 8 bytes
- **long double** → 8, 12, or 16 bytes

## Pointers

- **pointer** → 4 bytes (32-bit systems) / 8 bytes (64-bit systems)  
(endianness matters — it's just an integer address)

|                | endian | 0x1000 | 0x1001 | 0x1002 | 0x1003 |
|----------------|--------|--------|--------|--------|--------|
| Int a = 0x80   | Big    | 00     | 00     | 00     | 80     |
| Int a = 0x80   | Little | 80     | 00     | 00     | 00     |
| Byte a = 0xae  | Big    | ae     |        |        |        |
| Byte a = 0xae  | Little | ae     |        |        |        |
| Short = 0x1234 | Big    | 12     | 34     |        |        |
| Short = 0x1234 | Little | 34     | 12     |        |        |

Shift Left (<<) (defn: moves all bits to the left by n positions)

- Each shift by 1 = multiply by 2; empty bit position on the right are filled with 0s; bits that go past the left are discarded

E.g. 0000 0111 (0x07 = 7) << 2 0011 1000 (0x1C = 28)

Shift Right (>>); each shift right divides by 2; >> arithmetic shift keeps signed bit, >>> always fills left with 0s.

Random e.g.

Convert 354808 to hex: 0x000569F8 (pad for 4 bytes). Big-endian order: 00 05 69 F8

Two's complement e.g.

Number = -994 778

$2^{32} - 994\,778 = 4\,293\,972\,518$

$4\,293\,972\,518 = 0xFFFF0D226$  as bytes FF F0 D2 26

E.g. numbers  
Decimal to binary  
45: 0010 1101

Sign-magnitude ("normal")

+45 -> sign 0 + magnitude 010 1101 -> 0 010 1101 -> 0010 1101

- 45 -> sign 1 + magnitude 010 1101 -> 1 010 1101 -> 1010 1101

Addition doesn't really work that well here with negative and positive numbers.

One's complement:

+45: 0010 1101

-45: invert every bit -> 1101 0010

Notes: range (8-bit) -127...+127; has TWO zeros: 0000 0000 and 1111 1111. Addition uses "end-around carry"

|                 |                                                                                                           |
|-----------------|-----------------------------------------------------------------------------------------------------------|
| 45 + -45 = 0    | 0010 1101 + 1101 0010 = 1111 1111                                                                         |
| 50 + (-20) = 30 | 0011 0010 + 1110 1011 = 0001 1101 (carry out 1, then add 1 back in:)<br>0001 1101 + 0000 0001 = 0001 1110 |

Two's complement (rule: one's complement + 1, or "invert + 1")

+45: 0010 1101

- 45: 1101 0010 add 1 -> 1101 0011

Notes: range(8-bit) -128...127, only one zero (0000 0000).

|                 |                                                                         |
|-----------------|-------------------------------------------------------------------------|
| 45 + -45 = 0    | 0010 1101 + 1101 0011 = 1 0000 0000<br>(drop the header, so 0000 0000)  |
| 50 + (-20) = 30 | 0011 0100 + 1110 1100 = 1 0010 1110<br>(drop the header, so 0010 1110). |

Int to binary:

num = 42

binary\_representation = bin(num)

print(f"Binary representation of {num} is: {binary\_representation}")

Int to hex:

hex\_representation = hex(num)

print(f"Hexadecimal representation of {num} is: {hex\_representation}")

Bin to int:

binary\_str = "0b101010" # This is binary for 42

integer\_value = int(binary\_str, 2)

print(f"Integer value of {binary\_str} is: {integer\_value}")

Hex to int:

hex\_str = "0x2a" # This is hex for 42

integer\_value\_hex = int(hex\_str, 16)

print(f"Integer value of {hex\_str} is: {integer\_value\_hex}")